

MODEL DEVELOPMENT DOCUMENT (MDD)

ZOMATO AI RECOMMENDATION SYSTEM

Project Title: Zomato AI Recommendation System

Prepared By Name: kunal jain ,sonal jain

CERTIFICATE

This is to certify that the project entitled “**Zomato AI Recommendation System**” has been developed as part of the academic project work. The project uses the Zomato Bangalore Restaurants dataset and implements data preprocessing, feature preparation, similarity-based restaurant recommendation, and an interactive Streamlit interface.

The project demonstrates the practical application of Python programming, data analysis, machine learning concepts, feature engineering, similarity calculation, and web application development.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their valuable guidance and support throughout the development of the **Zomato AI Recommendation System**.

I am thankful to the contributors of the Zomato Bangalore Restaurants dataset available on Kaggle, which provided the restaurant information required for this project.

I would also like to acknowledge the technologies and libraries used during the development of the project, including Python, Pandas, Scikit-learn, and Streamlit. These tools helped us perform data processing, feature preparation, similarity calculations, and application development.

Finally, I would like to thank everyone who provided suggestions, feedback, and support during the completion of this project.

ABSTRACT

The **Zomato AI Recommendation System** is a Python-based restaurant recommendation application developed using the Zomato Bangalore Restaurants dataset.

The main purpose of this project is to help users discover restaurants that are similar to a restaurant selected by them. Bengaluru has a large number of restaurants offering different cuisines, restaurant types, price ranges, and services. Because of this large amount of information, manually finding suitable restaurants can be difficult.

To solve this problem, we developed a similarity-based restaurant recommendation system. The system takes restaurant information from the dataset and processes important attributes such as restaurant type, location, cuisines, dishes liked, and menu-related information.

The raw data is first loaded and preprocessed. Missing values and inconsistent formats are handled where required. Relevant restaurant attributes are then selected and combined to create a useful representation of each restaurant.

The recommendation component compares restaurant representations and calculates similarity between them. Based on the similarity scores, restaurants are ranked and the most similar restaurants are recommended to the user.

The complete system is implemented using Python and presented through a Streamlit web application. The application provides an easy-to-use interface where the user can select a restaurant and receive recommendations.

The project demonstrates how a real-world dataset can be transformed into a functional recommendation system using data preprocessing, feature engineering, similarity calculation, and interactive application development.

1. INTRODUCTION

Restaurant recommendation systems are becoming increasingly useful because users have a large number of restaurants to choose from.

A user may know that they like a particular restaurant but may not know which other restaurants offer similar food, restaurant types, or dining experiences.

The purpose of our project is to solve this problem using restaurant data.

We developed a **Zomato AI Recommendation System** using the Zomato Bangalore Restaurants dataset. The dataset contains information about restaurants in Bengaluru, including restaurant name, location, cuisine, rating, votes, restaurant type, online ordering, table booking, approximate cost, dishes liked, and menu information.

Instead of simply displaying the dataset to the user, we process the available information and use it to identify similar restaurants.

The project follows this general workflow:

Dataset → Data Processing → Feature Selection → Feature Preparation → Similarity Calculation → Recommendation → Streamlit Interface

The major focus of our project is therefore the transformation of raw restaurant data into useful recommendations.

2. PROBLEM STATEMENT

The Zomato Bangalore restaurant dataset contains a large number of restaurant records.

If a user wants to find a restaurant similar to one they already like, manually checking thousands of restaurant records is time-consuming.

For example, a user may select a restaurant that serves North Indian and Chinese cuisine. They may want recommendations for other restaurants offering similar cuisines and having similar restaurant characteristics.

The problem addressed by our project is therefore:

How can restaurant data be processed and compared automatically to recommend restaurants that are similar to a restaurant selected by the user?

To solve this problem, we developed a similarity-based recommendation system.

3. OBJECTIVES

The main objectives of our project are:

1. To understand the Zomato Bangalore Restaurants dataset.
2. To process and clean the raw restaurant data.
3. To identify useful restaurant attributes.
4. To prepare restaurant features for comparison.
5. To calculate similarity between restaurants.

6. To identify restaurants similar to the selected restaurant.
 7. To rank the similar restaurants.
 8. To display recommendations through a user-friendly interface.
 9. To develop the application using Python and Streamlit.
 10. To demonstrate the practical use of data science and machine learning concepts.
-

4. DATASET USED IN THE PROJECT

4.1 Dataset Source

We used the **Zomato Bangalore Restaurants Dataset** available on Kaggle.

The dataset contains restaurant information related to Bengaluru and is stored in the file:

zomato.csv

The commonly documented raw dataset contains:

- **51,717 records**
- **17 attributes**

The dataset represents Zomato restaurant information available up to 15 March 2019.

5. DATASET ATTRIBUTES

The dataset contains the following major attributes:

Attribute	Purpose
url	Restaurant URL
address	Restaurant address
name	Restaurant name
online_order	Availability of online ordering
book_table	Availability of table booking
rate	Restaurant rating
votes	Number of votes

Attribute	Purpose
phone	Restaurant contact number
location	Restaurant location
rest_type	Restaurant type
dish_liked	Dishes liked by customers
cuisines	Cuisines served
approx_cost(for two people)	Approximate cost for two people
reviews_list	Customer review information
menu_item	Menu information
listed_in(type)	Listing category
listed_in(city)	City listing

These attributes provide both descriptive and numerical information about restaurants.

6. UNDERSTANDING THE DATA USED BY OUR SYSTEM

Not every dataset column is equally important for recommendation.

Our recommendation system mainly focuses on restaurant characteristics that can help identify similarity.

For example:

Restaurant Type

rest_type tells us whether the restaurant is a cafe, casual dining restaurant, quick bite restaurant, etc.

Location

location identifies the area in which the restaurant is located.

Cuisine

cuisines describes the types of food served.

For example:

- North Indian

- South Indian
- Chinese
- Italian
- Mexican
- Thai

Dishes Liked

dish_liked provides information about dishes that customers liked.

Menu

menu_item provides information about menu offerings.

These attributes can provide useful information for comparing restaurants.

7. HOW WE STARTED THE PROJECT

The first step in our implementation was to obtain the dataset and load it into Python.

We used Pandas for data handling.

```
import pandas as pd
```

```
df = pd.read_csv("zomato.csv")
```

Here, `pd.read_csv()` reads the CSV file and stores the restaurant data in a Pandas DataFrame.

The DataFrame becomes the main data structure used throughout the project.

After loading the data, we can perform operations such as:

- Viewing columns
- Checking records
- Handling missing values
- Filtering restaurants
- Selecting features
- Creating new columns

This forms the foundation of the recommendation system.

8. DATA PREPROCESSING IMPLEMENTED IN THE PROJECT

Raw restaurant data cannot always be directly used by a recommendation algorithm.

There may be:

- Missing values
- Inconsistent text
- Non-numerical rating values
- Different formats
- Empty fields

Therefore, preprocessing is required.

8.1 Handling Missing Values

Some restaurant records contain missing information.

For example, fields such as cuisines, rest_type, location, and dish_liked may contain missing values.

For text-based processing, missing values can be replaced with empty strings.

Example:

```
df["cuisines"].fillna("")
```

The purpose of this operation is to ensure that missing values do not create errors when restaurant features are combined.

9. RATING PREPROCESSING

One important issue in the dataset is that the rate column is not always stored as a pure number.

Examples include:

4.1/5

4.5/5

3.8/5

NEW

-

Therefore, rating values need to be cleaned before numerical analysis.

For example:

4.1/5 → 4.1

Values such as NEW or - represent unavailable ratings and need to be handled appropriately.

This preprocessing makes the rating field easier to analyze and use for filtering.

10. COST PREPROCESSING

The dataset contains an approximate cost for two people.

Some values may appear with commas, for example:

1,000

1,500

2,000

These values need to be converted into an appropriate numerical representation before performing cost-based analysis or filtering.

This also allows the project to be extended later with a user budget option.

For example:

User Budget → Filter Restaurants → Apply Similarity → Display Recommendations

11. FEATURE SELECTION

Feature selection is one of the most important steps in our project.

We do not need every column in the dataset to calculate restaurant similarity.

Columns such as URL and phone number are useful for displaying or identifying restaurant information, but they are not necessarily meaningful similarity features.

For recommendation, we focus on descriptive restaurant information such as:

- Restaurant type
- Location
- Cuisine
- Dishes liked

- Menu information

These features describe what kind of restaurant it is and what it offers.

12. COMBINING RESTAURANT FEATURES

Instead of processing every text field separately, relevant restaurant attributes can be combined into one feature representation.

For example:

```
df["combined_features"] = (  
    df["rest_type"].fillna("") + " " +  
    df["location"].fillna("") + " " +  
    df["cuisines"].fillna("") + " " +  
    df["dish_liked"].fillna("") + " " +  
    df["menu_item"].fillna("")  
)
```

This creates a new column called:

`combined_features`

The purpose of this column is to create a single representation of the important characteristics of each restaurant.

For example, a restaurant may be represented approximately as:

Casual Dining Indiranagar North Indian Chinese Biryani

Another restaurant may be represented as:

Casual Dining Indiranagar North Indian Chinese Pasta

Because several characteristics are common, the two restaurants can be identified as similar.

13. WHY FEATURE COMBINATION IS IMPORTANT

Feature combination allows the system to compare restaurants based on multiple characteristics at the same time.

If we only used cuisine, two restaurants serving the same cuisine could be considered similar even if they were completely different restaurant types or locations.

By combining multiple characteristics, the recommendation system gets a broader representation of each restaurant.

For example:

Restaurant A

Cafe + Koramangala + Italian + Pizza

Restaurant B

Cafe + Koramangala + Italian + Pasta

The system can recognize that they have several common characteristics.

This makes feature engineering an important part of our project.

14. TEXT VECTORIZATION

Computers cannot directly perform mathematical comparison on raw text.

For example:

North Indian Chinese

cannot directly be used in a mathematical similarity formula.

Therefore, the combined restaurant features need to be converted into numerical vectors.

If the implementation uses TF-IDF, we use:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer(stop_words="english")
```

```
tfidf_matrix = tfidf.fit_transform(df["combined_features"])
```

The TF-IDF vectorizer converts the restaurant text into numerical representations.

Each restaurant is therefore represented as a vector that can be compared mathematically.

15. SIMILARITY CALCULATION

After converting restaurant features into numerical vectors, the next major step is to calculate similarity.

If cosine similarity is used in the implementation:

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
similarity_matrix = cosine_similarity(tfidf_matrix)
```

The result is a similarity matrix.

The matrix contains similarity values between restaurant pairs.

For example:

	Restaurant A	Restaurant B	Restaurant C
Restaurant A	1.00	0.82	0.20
Restaurant B	0.82	1.00	0.30
Restaurant C	0.20	0.30	1.00

The values shown above are only illustrative.

A higher similarity value means that the feature representations have more similarity.

16. COSINE SIMILARITY FORMULA

Cosine similarity measures the similarity between two vectors.

The formula is:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- A is the feature vector of one restaurant.
- B is the feature vector of another restaurant.
- $A \cdot B$ is their dot product.
- $\|A\|$ and $\|B\|$ are the magnitudes of the vectors.

The similarity score allows the system to rank restaurants according to how closely their feature representations match.

17. HOW OUR RECOMMENDATION LOGIC WORKS

The main recommendation process is:

Step 1

The user selects a restaurant.

Step 2

The system searches for that restaurant in the DataFrame.

Step 3

The system obtains the index of the selected restaurant.

Step 4

The corresponding similarity scores are retrieved.

Step 5

The restaurants are sorted according to similarity.

Step 6

The selected restaurant itself is excluded.

Step 7

The top similar restaurants are selected.

Step 8

The recommendations are displayed in the Streamlit application.

Therefore:

User Selection → Restaurant Index → Similarity Scores → Sorting → Top Similar Restaurants → Display

18. WHY WE REMOVE THE SELECTED RESTAURANT

This is an important part of the recommendation logic.

A restaurant will generally have the highest similarity with itself.

For example:

Restaurant A → Restaurant A = 1.00

If we simply sort all similarity values, Restaurant A would appear as the first recommendation.

However, the user already selected Restaurant A.

Therefore, our recommendation logic should skip or remove the selected restaurant and return other restaurants.

This ensures that the output contains actual recommendations rather than repeating the user's selection.

19. RANKING THE RECOMMENDATIONS

After similarity values are obtained, they are sorted.

Conceptually:

Selected Restaurant



Similarity Scores



Sort from Highest to Lowest



Remove Selected Restaurant



Select Top Results

For example:

Restaurant	Similarity
------------	------------

Restaurant B	0.89
--------------	------

Restaurant C	0.84
--------------	------

Restaurant D	0.79
--------------	------

Restaurant E	0.74
--------------	------

The restaurants with higher similarity are placed earlier in the recommendation list.

20. STREAMLIT APPLICATION DEVELOPMENT

After developing the recommendation logic, we needed a simple interface through which users could interact with the system.

For this purpose, we used **Streamlit**.

Streamlit allows Python code to be presented as an interactive web application.

The user does not need to interact with the underlying Python code.

Instead, they interact with:

- Restaurant selection
- Buttons
- Input elements
- Recommendation results

The application connects the frontend with the recommendation logic.

21. APPLICATION TITLE AND DESIGN

We also customized the appearance of the application.

For example:

```
st.markdown(
    '<div class="main-title">🍽️ Zomato AI Recommendation</div>',
    unsafe_allow_html=True
)
```

This displays the main title of the application.

CSS styling can be used to modify:

- Font size
- Alignment
- Spacing
- Appearance

This improves the presentation of the application.

22. USER INTERACTION

The application is designed around a simple user workflow.

The user opens the application and selects a restaurant.

For example:

Zomato AI Recommendation



Select Restaurant



Get Recommendations



Recommended Restaurants

After selection, the backend recommendation logic processes the selected restaurant and finds similar restaurants.

The user therefore does not need to understand the mathematical calculations occurring in the backend.

23. COMPLETE SYSTEM ARCHITECTURE

Our project can be divided into several layers.

Data Layer

Contains:

zomato.csv

Processing Layer

Performs:

- Data loading
- Cleaning
- Missing-value handling
- Feature preparation

Feature Layer

Creates the restaurant feature representation.

Similarity Layer

Calculates restaurant similarity.

Recommendation Layer

Ranks and selects similar restaurants.

Presentation Layer

Displays the recommendations using Streamlit.

The architecture is:

Zomato Dataset



Pandas Data Processing



Feature Engineering



Feature Vectorization



Similarity Matrix



Recommendation Logic



Streamlit Interface



User

24. COMPLETE ALGORITHM

Input

A restaurant selected by the user.

Output

A list of similar restaurants.

Algorithm

1. Load the Zomato restaurant dataset.
2. Store the data in a Pandas DataFrame.
3. Inspect the dataset structure.
4. Handle missing values.

5. Clean relevant fields.
 6. Process rating and cost fields where required.
 7. Select relevant recommendation features.
 8. Combine selected restaurant attributes.
 9. Convert the combined representation into numerical vectors.
 10. Calculate similarity between restaurant vectors.
 11. Accept a restaurant selected by the user.
 12. Find its corresponding DataFrame index.
 13. Retrieve its similarity scores.
 14. Sort the similarity scores.
 15. Remove the selected restaurant.
 16. Select the top similar restaurants.
 17. Display the results through Streamlit.
-

25. HOW THE CODE SOLVES THE ORIGINAL PROBLEM

The project starts with a problem:

There are many restaurants, and the user needs relevant alternatives.

Our code solves this step-by-step.

Raw Data Problem

The restaurant information is stored in a CSV file.

Solution:

Pandas is used to load and organize the data.

Data Quality Problem

Some fields are missing or inconsistent.

Solution:

Missing values and required formats are handled during preprocessing.

Comparison Problem

Restaurant information is stored as text and categories.

Solution:

Relevant attributes are combined and converted into a numerical representation.

Similarity Problem

The system needs to determine which restaurants are similar.

Solution:

A similarity calculation is applied to the restaurant feature vectors.

Recommendation Problem

The system needs to select the most similar restaurants.

Solution:

Similarity scores are sorted and the top results are selected.

User Experience Problem

The recommendation logic needs to be accessible to a normal user.

Solution:

Streamlit is used to provide an interactive interface.

Thus, every major stage of the code contributes to solving one part of the original problem.

26. PROJECT MODULES

The project can be divided into the following modules.

Module 1 – Dataset Loading

Loads the CSV dataset into Python.

Module 2 – Data Preprocessing

Handles missing values and inconsistent data.

Module 3 – Feature Engineering

Selects and combines relevant restaurant attributes.

Module 4 – Vectorization

Converts textual restaurant information into numerical vectors.

Module 5 – Similarity Engine

Calculates similarity between restaurant representations.

Module 6 – Recommendation Engine

Ranks similar restaurants and selects the required results.

Module 7 – Streamlit Interface

Provides the user interface and displays recommendations.

27. INPUT, PROCESSING AND OUTPUT

Input

The main user input is:

Restaurant selected by the user

The dataset itself provides the restaurant information used by the recommendation algorithm.

Processing

The system performs:

- Data loading
- Data cleaning
- Feature selection
- Feature combination
- Vectorization
- Similarity calculation
- Ranking

Output

The final output is:

A list of restaurants similar to the selected restaurant.

28. TESTING

The application should be tested using different scenarios.

Test Case	Input	Expected Result
Dataset Loading	zomato.csv	Dataset loads successfully

Test Case	Input	Expected Result
Restaurant Selection	Valid restaurant	Restaurant is selected
Recommendation	Selected restaurant	Similar restaurants are displayed
Missing Data	Restaurant with missing fields	Application continues without failure
Invalid Selection	Invalid restaurant	Appropriate message is displayed
UI Test	User interaction	Interface responds correctly

Testing helps verify that both the backend recommendation logic and frontend application work correctly.

29. RESULTS AND DISCUSSION

The developed system successfully demonstrates the concept of similarity-based restaurant recommendation.

The important result of the project is that a user can select a restaurant and receive other restaurants based on similarity between their available characteristics.

The recommendation system does not simply display random restaurants.

Instead, it follows a processing pipeline:

Restaurant Information → Features → Numerical Representation → Similarity → Ranking → Recommendation

The system is particularly useful for demonstrating how restaurant characteristics can be converted into machine-readable information and used for recommendation.

The Streamlit interface makes the result easier for users to access.

30. ADVANTAGES

The main advantages of our implementation are:

1. It uses a real-world restaurant dataset.
2. It automates restaurant recommendation.
3. It uses restaurant characteristics to identify similar restaurants.
4. It reduces the need for manual searching.
5. It provides an interactive interface.

6. It combines data processing with machine learning concepts.
 7. It can be extended with additional filtering features.
 8. It provides a practical demonstration of a recommendation system.
-

31. LIMITATIONS

There are some limitations in the current implementation.

Historical Dataset

The dataset represents a historical snapshot of restaurant information and may not represent current restaurant status.

Limited Personalization

The current recommendation approach primarily uses restaurant attributes and does not necessarily learn a complete personal profile of each user.

Missing Data

Missing restaurant information can affect the quality of feature representation.

New Restaurants

A newly added restaurant with limited information may be difficult to recommend accurately.

Bengaluru Focus

The dataset is focused on Bengaluru restaurants, so the current system is not directly designed for all cities.

32. FUTURE SCOPE

The project can be improved in several ways.

User Preference

The system can allow users to enter their preferred cuisine, budget, location, and minimum rating.

Personalized Recommendation

User history can be stored and used to provide more personalized results.

Location-Based Recommendation

The system can recommend restaurants based on the user's selected location.

Budget-Based Recommendation

Restaurants can be filtered according to the user's budget.

Review Sentiment Analysis

Customer reviews can be processed using Natural Language Processing to understand customer sentiment.

Hybrid Recommendation

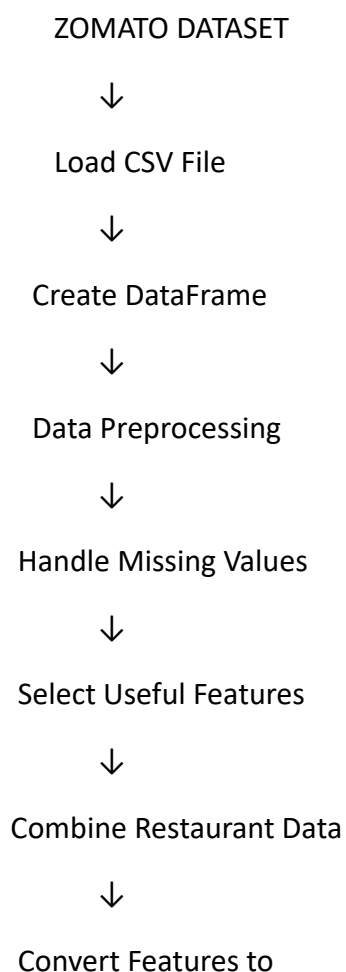
Content similarity can be combined with ratings, popularity, user history, and preferences.

Online Deployment

The Streamlit application can be deployed online so that users can access it through a web browser.

33. FINAL PROJECT WORKFLOW

The complete implementation can be summarized as:



Numerical Representation



Calculate Similarity



User Selects Restaurant



Find Selected Restaurant



Retrieve Similarity Scores



Sort Similar Restaurants



Remove Selected Restaurant



Select Top Recommendations



Display Using Streamlit

34. CONCLUSION

The **Zomato AI Recommendation System** is a practical implementation of a restaurant recommendation application using Python and Streamlit.

The project starts with the Zomato Bangalore Restaurants dataset and processes the restaurant information to make it suitable for recommendation.

The important part of our implementation is the use of restaurant features such as restaurant type, location, cuisines, dishes liked, and menu-related information.

These features are prepared and converted into a form that can be compared mathematically. Similarity is then calculated between restaurants, and the restaurants with higher similarity are selected as recommendations.

The Streamlit interface connects the recommendation logic with the user. The user simply selects a restaurant, while the backend performs the data processing, similarity calculation, ranking, and recommendation generation.

Therefore, our project demonstrates the complete process of converting raw restaurant data into a working recommendation application.

The overall concept can be summarized as:

Raw Restaurant Data

→ **Data Cleaning**

→ **Feature Engineering**

→ **Similarity Calculation**

→ **Restaurant Ranking**

→ **Recommendation**

→ **Interactive Streamlit Application**

The project can be further enhanced by adding user preferences, budget filtering, location-based recommendations, review sentiment analysis, user history, and hybrid recommendation techniques.

REFERENCES

1. Himanshu Poddar, **Zomato Bangalore Restaurants Dataset**, Kaggle.
2. Zomato Bangalore restaurant dataset analysis and documentation resources.
3. Python Documentation.
4. Pandas Documentation.
5. Scikit-learn Documentation.
6. Streamlit Documentation.

TECHNOLOGY SUMMARY

Component	Technology
Programming Language	Python
Data Processing	Pandas

Component	Technology
Machine Learning	Scikit-learn
Recommendation Method	Similarity-Based Recommendation
Web Framework	Streamlit
Dataset	Zomato Bangalore Restaurants
Dataset File	zomato.csv
Data Format	CSV